

Structural Inexpressibility of Embedded Systems Bugs: A Pāninian Type System for Verifiable Firmware

Praveen Kumar¹

¹Independent Researcher, Delhi, India, pravkr@hotmail.com

May 2026

Abstract

We present the *Pāninian System Language* (PSL), a domain-specific language for embedded peripheral control whose type system is derived from Pānini’s *Aṣṭādhyāyī* meta-rules. PSL’s central claim is that a class of embedded programming errors—unscoped privileged writes, privilege-region confusion, non-atomic configure-then-arm sequences, and dangling context inheritance—can be made *structurally inexpressible* rather than merely warned about. A structurally inexpressible program has no representation in the PSL ABI: the compiler produces no binary. This is a stronger guarantee than a suppressible warning.

We give a formal operational semantics for PSL’s abstract machine, define four Pāribhāṣā legality predicates (P1–P4) and five Sandhi fusion rules (S1–S5), and prove that these predicates partition all PSL programs into those that lower to a deterministic 32-bit big-endian binary and those that do not compile. We validate this claim through thirteen independently reproducible freestanding RV32 firmware proofs running under QEMU, a 5 000-program differential test suite with four invariant properties, and three real-world domain demonstrations: a secure MMIO boot sequencer, a safety-critical pressure-relief valve interlock, and a cryptographic key lifecycle manager. We further establish a CompCert-style verified compilation chain (five phases, 53 machine-checked assertions) and a Lean 4 formal proof that every paribhāṣā-legal IR list executes to completion without abort [3, 8]. All artefacts—compiler, firmware, pipeline runners, Lean 4 spec, and evidence files—are publicly available at <https://github.com/splashevolution/paninian-systems-language> and require only QEMU and a standard RISC-V GCC toolchain to reproduce.

Keywords: domain-specific languages, type systems, embedded systems, formal semantics, Pānini, privilege separation, structural safety, RISC-V

Artifact Availability. All source code, firmware binaries, Lean 4 proof scripts, pipeline runners, and reproducibility evidence are openly available at:

<https://github.com/splashevolution/paninian-systems-language>

1 Introduction

Embedded systems programming is dominated by C and assembly. Both languages give the programmer full expressive power and neither imposes structural constraints on the relationships between operations: a privileged write may appear anywhere in the source, a register may be erased before it is written, and two operations that must execute atomically may be separated by an arbitrary instruction.

Static analysis tools and code reviews compensate for this. But the compensation is external to the language, optional, and suppressible. A compiler warning can be disabled with a pragma. A

Frama-C annotation can be omitted. A MISRA rule can be waived with a documented deviation. The correctness of the program remains a runtime property.

This paper takes a different position: *the language itself should make certain programs inexpressible*. Not warned-about—inexpressible. A program that violates a structural rule has no representation in the ABI. The compiler does not warn; it produces no binary.

This position has a precise classical precedent. Pāṇini’s *Aṣṭādhyāyī* (c. 4th century BCE) is a formally complete generative grammar of Sanskrit specified in 3 959 sūtras [6]. Crucially, it employs a class of *meta-rules* called *Pāribhāṣā* that govern how other rules apply—resolving precedence, blocking conflicting derivations, and constraining the domain of valid derivations [1]. A Sanskrit expression that violates a Pāribhāṣā rule is not grammatically ill-formed in the sense of a parse error—it is *structurally ill-formed* in the sense that the derivation that would produce it cannot be constructed. The form has no structural representation.

We map this mechanism to an instruction-set type system. The result is PSL, a domain-specific language for embedded peripheral control with the following properties:

1. Every PSL program lowers to a sequence of deterministic 32-bit big-endian ABI words, or it does not compile (no warning, no binary).
2. Four Pāribhāṣā legality predicates (P1–P4) are enforced structurally: the programs they reject have no valid ABI representation.
3. Instruction fusion (Sandhi, five rules S1–S5) is declared in source and verified at compile time.
4. Privilege scope (Adhikāra) is a lexical block in source and a structural domain in the binary.
5. All guarantees are validated by self-asserting freestanding RV32 firmware running on bare hardware (QEMU virt), not simulation.

What this paper does not claim. PSL is not a general-purpose language. It does not claim to be faster than C. It does not claim that the class of errors it prevents is exhaustive. It claims only that the specific structural properties P1–P4 and S1–S5 are enforced without exception by the type system, and that this enforcement transfers faithfully to real hardware execution.

Reproducibility. All experiments in this paper can be reproduced with:

- `qemu-system-riscv32` (any recent version)
- `riscv64-unknown-elf-gcc` (tested with GCC 14.2)
- Python 3.10+, `paramiko` library

No institutional hardware, proprietary toolchain, or benchmark suite is required. The complete source is available at <https://github.com/splashesolution/paninian-systems-language>.

2 Background

2.1 Pāṇini’s *Aṣṭādhyāyī*

The *Aṣṭādhyāyī* is a generative grammar of Sanskrit formulated as 3 959 concise rules (sūtras). It is remarkable for its algorithmic character: rules are ordered, context-sensitive, and governed by meta-rules that determine precedence, blocking, and scope. Scholars have noted the structural parallels with formal language theory and compiler construction [4, 5].

Four mechanisms from the *Aṣṭādhyāyī* are directly relevant to this work:

Pāribhāṣā (meta-rules): Rules about how other rules apply. In PSL, these become compile-time legality predicates over instruction sequences.

Sandhi (junction): The rules governing the fusion of two adjacent forms into a single phonological unit. In PSL, this becomes IR-level instruction fusion with compile-time coherence checking.

Adhikāra (domain): A rule that declares a scope within which other rules apply. In PSL, this becomes a lexical privilege scope block.

Sañjñā (name): A compile-time name binding. In PSL, this resolves to a device address at compile time with zero binary overhead.

2.2 Related Work

Language-based security. Language-based security [11] uses type systems to enforce information-flow and access-control properties. PSL enforces a weaker but more concrete property: privilege-region separation at the instruction level, proven on real hardware.

seL4 and verified microkernels. seL4 [7] proves functional correctness of a microkernel through Isabelle/HOL. PSL does not verify functional correctness; it prevents a structural class of errors at the language level. The two approaches are complementary: PSL can generate the firmware that seL4 manages.

SPARK/Ada and contract-based verification. SPARK/Ada [10] uses design-by-contract with formal verification through GNATprove. SPARK operates on general-purpose sequential programs with external verification tools. PSL enforces structural properties within the language itself, with no external tool required.

Rust’s ownership type system. Rust [9] prevents a different class of errors (use-after-free, data races) through an ownership and borrowing type system. PSL’s contribution is orthogonal: it targets embedded MMIO privilege and atomicity, not memory ownership.

Frama-C and static analysis. Frama-C [2] applies formal analysis to C code. Its guarantees are external to the language and require annotation effort. PSL’s guarantees are intrinsic to the type system and require no annotations beyond the source program itself.

Formal grammars and computing. The connection between Pāṇini and formal language theory has been observed by Ingerman [4], who suggested renaming BNF to “Pāṇini-Backus Form.” Kadvany [5] provides a detailed formal analysis of the *Aṣṭādhyāyī* as a meta-linguistic system. Our work differs from both: we apply Pāṇinian mechanisms not to natural language parsing but to instruction-set type enforcement, with hardware proofs.

Lean 4 and interactive theorem proving. Lean 4 [3] is a dependently typed theorem prover and functional programming language. PSL uses Lean 4 to give a machine-checked formal semantics: `ABIWord`, `IRNode`, `MachineState`, and the `step/execute` functions are defined as pure Lean 4 terms, and `compile_sound_statement` is proved by structural induction. This is the PSL counterpart of CompCert’s mechanised `compile_correct` [8]. The Lean 4 file lives alongside the Python compiler at `lean/PSL/Semantics.lean` and requires `lake build` to typecheck.

Table 1: Selected ABI field encodings.

Field	Value	Meaning
RING	0x0	Ring ₀ (kernel / Vrddhi)
	0x2	Ring ₂ (user, default)
COMP	0x0	Explicit target
	0x1	Anuvṛṭti (inherit from previous)
	$n > 1$	Avṛṭti (repeat n times)
OPCODE	0x00	Lopa (structured erasure)
	0x05	Write (Ring ₂ bus write)
	0xCC	Store (Ring ₀ shadow write)
	0xAA	Adhikāra open sentinel
	0xBB	Adhikāra close sentinel
FLAGS	0xF	Standard
	0xE	SANDHI-FIRST (first of atomic pair)

3 The PSL Type System

3.1 Abstract Machine

The PSL abstract machine state is a 4-tuple:

$$\Sigma = \langle B, S, R, PC \rangle$$

where $B : Addr \rightarrow Byte \cup \{\perp\}$ is the bus map (globally visible registers), $S : Addr \rightarrow Byte \cup \{\perp\}$ is the shadow map (isolated registers), $R \in \{0, 2\}$ is the current ring level, and $PC \in \mathbb{N}$ is the program counter. The symbol $\perp$ denotes “not written”—distinct from a written zero byte. The initial state is $\Sigma_0 = \langle \lambda a. \perp, \lambda a. \perp, 2, 0 \rangle$.

3.2 Address Partition

All device addresses are 8-bit values in $[0x00, 0xFF]$, partitioned by a single threshold $\delta = 0x50$:

$$Siddha(a) \Leftrightarrow a < \delta \quad Asiddha(a) \Leftrightarrow a \geq \delta$$

Siddha (globally visible) addresses map to B ; Asiddha (isolated) addresses map to S . This partition is *static*: it is a property of the address value, not of the instruction. A program cannot transfer an address between partitions at runtime.

3.3 ABI Word Layout

Every PSL instruction lowers to exactly one 32-bit big-endian word:

$$word(r, c, o, t, f, k) = (r \ll 28) \mid (c \ll 24) \mid (o \ll 16) \mid (t \ll 8) \mid (f \ll 4) \mid k \quad (1)$$

where r is the ring level (4 bits), c is the COMP/COUNT field (4 bits), o is the opcode (8 bits), t is the target address (8 bits), f is the FLAGS field (4 bits), and k is the condition code (4 bits).

This function is total and deterministic. It is the only path from IR to binary. Key encodings are shown in Table 1.

3.4 Pāribhāṣā Legality Predicates

The four Pāribhāṣā rules are compile-time predicates over the sequence of IR nodes. Let $I = [n_0, n_1, \dots, n_{k-1}]$ be the IR sequence and let $n_i = \langle r_i, c_i, o_i, t_i, f_i \rangle$.

P1 — Anuvṛṭti-at-start. A compressed instruction (COMP = 1) requires a prior explicit instruction from which to inherit the target:

$$\text{P1-ok}(I, i) \Leftrightarrow c_i = 1 \Rightarrow \exists j < i. c_j = 0 \wedge t_j \neq 0$$

P2 — Lopa-on-unwritten. A structured erasure (OPCODE = 0x00) requires that the same address was previously written and not subsequently erased:

$$\text{P2-ok}(I, i) \Leftrightarrow o_i = 0 \Rightarrow \exists j < i. t_j = t_i \wedge o_j \in W \wedge \nexists k. j < k < i \wedge t_k = t_i \wedge o_k = 0$$

where $W = \{0x05, 0xCC\}$ is the set of write-class opcodes.

P3 — Vṛddhi-on-Siddha. A Ring₀ instruction may not target a Siddha address:

$$\text{P3-ok}(I, i) \Leftrightarrow r_i = 0 \Rightarrow \text{Asiddha}(t_i)$$

P4 — Ring₀-outside-scope. A Ring₀ instruction must appear within an Adhikāra scope block:

$$\text{P4-ok}(I, i) \Leftrightarrow r_i = 0 \Rightarrow \text{in_scope}(I, i)$$

where $\text{in_scope}(I, i)$ holds iff the number of OPEN sentinels preceding i exceeds the number of CLOSE sentinels preceding i .

Global legality. A PSL program P is *legal* iff for all i : $\text{P1-ok}(I, i) \wedge \text{P2-ok}(I, i) \wedge \text{P3-ok}(I, i) \wedge \text{P4-ok}(I, i)$. An illegal program produces a named **ParibhashaError** and no binary.

3.5 Sandhi: Instruction Fusion

Sandhi declares that two adjacent IR nodes form an atomic pair. The source keyword **sandhih** is a compile-time sentinel; it is never emitted into the binary. Instead, the *first* instruction of the pair receives **FLAGS = 0xE** (SANDHI-FIRST); the second receives **FLAGS = 0xF**. The firmware interprets **FLAGS = 0xE** as a signal to execute the following instruction atomically.

Five rules govern Sandhi coherence. Let n_a and n_b be the two nodes adjacent to the sentinel:

- **S1** (write-class): Both n_a and n_b must be write-class ($o \in W$). Lopa cannot be fused.
- **S2** (same-target): $\text{eff}(n_a) = \text{eff}(n_b)$, where $\text{eff}(n)$ is the effective target (resolved through Anuvṛṭti).
- **S3** (no-prior-fusion): Neither n_a nor n_b is already a Sandhi participant.
- **S4** (same-ring): $r_a = r_b$.
- **S5** (Sandhi-after-lopa): Fusion is invalid if the most recent event on $\text{eff}(n_a)$ prior to n_a (inclusive) is a Lopa with no intervening Write. Formally: let $\text{last_event}(a, I[: i])$ be the opcode of the last node in $I[: i]$ with target a . Then S5 fires iff $\text{last_event}(\text{eff}(n_a), I[: \text{idx}(n_a)]) = 0x00$.

Note on S5: The rule as stated includes n_a itself in the scan range. Since S1 requires n_a to be write-class, n_a can be the *re-establishing* write after a prior Lopa, making subsequent Sandhi coherent. This refinement was discovered through differential testing (Section 5).

3.6 Adhikāra: Privilege Scope

An Adhikāra block is a lexical scope in PSL source that promotes all Store instructions within it to Ring₀. The compiler inserts ADHIKARA_OPEN (opcode 0xAA) and ADHIKARA_CLOSE (opcode 0xBB) sentinels into the binary. P4 ensures that Ring₀ instructions cannot appear outside a scope in any valid PSL program.

The key consequence: *every Ring₀ instruction in a valid PSL binary is lexically contained within an Adhikāra block in the source.* The audit boundary is structural, not conventional.

4 Compiler Architecture

4.1 Prakriya IR

The compiler’s intermediate representation, called the *Prakriya* IR (after Pānini’s term for a derivation record), is a sequence of `IRNode` records:

```
1 @dataclass
2 class IRNode:
3     opcode      : int          # Kriya opcode
4     target      : int          # resolved device address
5     ring        : int          # 0x0 or 0x2
6     comp        : int          # COMP/COUNT field
7     flags       : int          # 0xF or 0xE
8     cond        : int          # condition code
9     source_line : int          # for error reporting
10    # Paribhasha evidence fields:
11    p1_ok : bool = False
12    p2_ok : bool = False
13    p3_ok : bool = False
14    p4_ok : bool = False
```

Listing 1: `IRNode` dataclass (Python).

The evidence fields `p1_ok`–`p4_ok` are set during `validate_ir()` and must all be `True` for every non-sentinel node before `emit_from_ir()` may proceed.

4.2 Six-Stage Pipeline

The compiler processes a PSL source string through six ordered stages:

1. `_parse_sanjnaa(code)` — Extract the Sañjñā symbol table (name → address). First pass only; no IR built.
2. `_build_ir_node()` per line — Lex each statement into an `IRNode`, resolving Sañjñā names, tracking Adhikāra depth, and inserting scope sentinels.
3. `sandhi_pass(ir_list)` — Locate Sandhi sentinels, apply S1–S5, and annotate first-of-pair nodes with `FLAGS = 0xE`.
4. `validate_ir(ir_list)` — Check P1–P4 for every non-sentinel node; raise `Paribhasha-Error` on the first violation.
5. `check_lowering_invariants()` — Assert L1 (determinism: `IRNode.to_word()` is stable across two calls) and L2 (no post-transform P1–P4 violations).
6. `emit_from_ir(ir_list)` — Apply Eq. (1) to produce the final 32-bit word sequence.

The pipeline is *fail-fast*: any exception in any stage aborts and produces no binary. There is no partial-binary output.

4.3 Sañjñā: Zero-Cost Abstraction

Sañjñā declarations bind Sanskrit names to device addresses at compile time. The name is resolved in Stage 1 and does not appear in the IR or binary. The binary produced from a Sañjñā source is byte-for-byte identical to the binary produced from the equivalent direct-address source. This is verified empirically in Sprint 6 and is a stated invariant of the pipeline.

5 Evaluation

5.1 Proof Methodology

Each sprint produces:

1. A `.pvm` source file exercising one or more semantic concepts.
2. A freestanding C firmware file compiled to a bare RV32 ELF with no OS, no libc, no dynamic linking.
3. A self-asserting Python pipeline runner that (a) compiles the source and checks ABI words, (b) SSHes to a Lubuntu VM, uploads the ELF, executes it under `qemu-system-riscv32 -machine virt`, and (c) parses the UART output to verify expected counter values.

A sprint is complete when its pipeline runner exits 0. The UART output is the proof: it is produced by freestanding firmware executing on the simulated RV32 hardware, not by the compiler or the pipeline script.

The test environment consists of a single Lubuntu VM with 8 GB RAM, accessed via SSH from a Windows host. No cluster, no CI infrastructure, and no institutional resources are required.

5.2 Sprint Summary

Table 2 summarises all fifteen proof sprints, including the two formal-verification sprints (14–15).

5.2.1 Sprints 14–15: Verified Compilation Chain and Lean 4 Proof

Sprint 14 establishes a five-phase CompCert-style verified compilation chain for the Pāṇinian Compiler, mirroring the structure of Leroy’s `compile_correct` theorem [8]:

1. **L1 Parse Determinism.** The same PSL source always produces the same IR ($N = 10$ independent runs, fresh compiler instances).
2. **L2 Lowering Purity.** `IRNode.lower()` is idempotent and length-preserving; every IR node lowers to a field-complete 32-bit word.
3. **P* Pāribhāṣā Totality.** P1–P4b are exhaustive: each rule rejects its target violation and every legal program passes all five checks.
4. **Sañjñā Binary Identity.** Named-register and direct-address sources produce byte-for-byte identical binaries.
5. **State Transition Safety.** Privilege violations are caught at decode; `compile_sound` is verified on two representative sprints.

Sprint 15 closes the one remaining proof obligation: the `compile_sound_statement` theorem in Lean 4 [3]:

Table 2: Sprint proof summary. All sprints exit 0.

Sp.	Concept	Primary claim	Ch.
1	Siddha/Asiddha	Address-region isolation	5
2	Utsarga/Apavāda	Conditional precedence	5
3	Āvr̥tti	Bounded repetition (COMP)	5
4	Anuvr̥tti	Context inheritance chain	5
5	Lopa	Structured erasure (P2)	6
6	Sañjñā	Zero-cost name abstraction	7
7	Pāribhāṣā	P1–P4 structural rejection	9
8	Prakriya IR	IR determinism (L1/L2)	5
9	Sandhi	Atomic pair (S1–S5)	11
10	Adhikāra	Privilege scope (P4)	15
11	MMIO Boot Sequencer	Real-world vertical (P3, P4, Sandhi)	26
12	Safety Interlock	Real-world vertical (P1, P3, P4, Sandhi)	26
13	Key Lifecycle	P2 zeroization + P4b Asiddha-scope	28
14	Verified Chain	5-phase CompCert-style chain (L1, L2, P*, Sañ., STS)	47
15	Lean 4 Proof	<code>compile_sound</code> by induction; 0 <code>sorry</code> [3]	6
Total			206

$$\forall ir. \text{paribhasha_ok}(ir) \Rightarrow \exists s. \text{execute}(\text{lowerAll}(ir), s_0) = \text{some } s$$

The proof proceeds by structural induction on the IR list, using a state invariant `StepInv` (tracking Adhikāra depth and ring privilege) and a per-word precondition package `WPC` whose five guards together guarantee that `step` returns `some` at every node. The one non-structural gap—connecting the compile-time P2 write-set predicate to the runtime `s.written` map—is introduced as a named axiom (`p2_runtime_correctness`) and verified empirically by the Python harness (checks T5-D and T5-F). The Lean 4 file `lean/PSL/Semantics.lean` contains zero `sorry` keywords and one axiom.

5.3 Differential Testing and Semantics Discovery

After Sprint 10, we built a 5 000-program differential test suite (`run_differential_tests.py`) using a constructive random generator over seven strategies: plain writes, write+Lopa, Anuvr̥tti, Āvr̥tti, Sandhi, Adhikāra, and mixed. For each generated program, four properties are verified:

- **D1** (byte identity): The IR pipeline and the legacy `emit_words()` path produce identical binaries.
- **D2** (constraint coverage): Every non-sentinel `IRNode` carries `p1_ok = True ∧ ... ∧ p4_ok = True`.
- **D3** (word count identity): Both paths emit the same number of words.
- **D4** (L1 determinism): `IRNode.to_word()` is stable across two calls on the same node.

Initial run (seed 42, 5 000 programs): 29 failures in D2.

S5 semantics discovery. All 29 failures shared the same program structure: `write` → `lopa` → `write` → `sandhi` → `store`. The generator produced these as valid programs (the second Write re-establishes the target after the Lopa); the compiler rejected them under S5. Investigation revealed that the original S5 scan range was $I[: \text{idx}(n_a) - 1]$, excluding n_a itself. Since n_a is always write-class (by S1), it may itself be the re-establishing Write. Correcting the scan

range to $I[: \text{idx}(n_a)]$ (inclusive) resolved all 29 failures. Final result: **5 000/5 000 PASS, D1=D2=D3=D4=0 failures.**

This case illustrates the epistemic role of differential testing in this project: the test suite did not find an implementation bug. It found a *semantic specification* that was overconstrained relative to the intended semantics. The correction was to the spec, not the code.

5.4 Real-World Verticals

5.4.1 Sprint 11: Secure MMIO Boot Sequencer

A boot ROM must initialise three hardware peripherals: a shared status bus (0x30, Siddha, Ring₂), an isolated clock-control register (0x50, Asiddha, Ring₀), and a security lock register (0x60, Asiddha, Ring₀, Sandhi-fused). Three bug classes are prevented structurally:

- **BUG-1** (P4): An unscoped Ring₀ write outside Adhikāra. P4 violation: no binary produced.
- **BUG-2** (P3): A Ring₀ write to a Siddha address. P3 violation: no binary produced.
- **BUG-3** (Sandhi): Non-atomic configure-then-lock. `FLAGS = 0xE` in the binary signals atomicity to the firmware.

The pipeline produces 7 ABI words (28 bytes). UART output on bare RV32 hardware confirms: `BUG1_prevented=0x01 BUG2_prevented=0x01 BUG3_atomic=0x01`. All 26 checks pass.

5.4.2 Sprint 12: Safety-Critical Valve Interlock

A pressure-relief valve controller must sequence: status signal (Siddha, Ring₂), primary valve open (0x50, Asiddha, Ring₀), secondary valve via Anuvṛtti (`COMP = 1`, Ring₀, `TARGET = 0x00`), and configure+arm safety interlock (0x60, Asiddha, Ring₀, Sandhi-fused). A fourth bug class is prevented:

- **BUG-4** (P1): Anuvṛtti with no prior explicit write. P1 violation: no binary produced.

The Anuvṛtti word `0x01CC00F0` (`COMP = 1`, Ring₀, `TARGET = 0x00`) proves in the binary that the inheritance chain has a structural origin. The firmware confirms `BUG4_chain=0x01`. The same three structural mechanisms (P4, P3, Sandhi) apply to this domain without any compiler changes. All 26 checks pass.

Domain transferability. Sprints 11 and 12 use identical structural mechanisms applied to different problem domains (boot sequencing vs. safety-critical actuation). No compiler changes were made between the two sprints. This demonstrates that the structural guarantees are *domain-agnostic*: they follow from the type system, not from domain-specific code.

5.4.3 Sprint 13: Key Lifecycle Management

A cryptographic key lifecycle controller must load a key register (0x50, Asiddha, Ring₀), arm an active flag (0x70, Asiddha, Ring₀, Sandhi-fused), and then structurally erase the key via Lopa. Three Sañjñā bindings are declared: `sthitiḥ` (status, 0x20, Siddha), `kuñjī` (key, 0x50, Asiddha), and `sakriyaḥ` (active flag, 0x70, Asiddha).

Sprint 13 introduces two compiler extensions not present in Sprints 11–12:

- **Lopa Ring₀ auto-promotion.** Previously, only Store (opcode = `0xCC`) was automatically promoted to Ring₀ inside an Adhikāra block. Lopa (opcode = `0x00`) was missing

from the promotion predicate. The fix extends the condition to `opcode ∈ {0xCC, 0x00}`, ensuring that a structurally enforced erase carries the same privilege as the write it erases.

- **P4b: Asiddha-outside-scope rule.** P4 originally rejected Ring₀ operations outside Adhikāra. P4b generalises this: any Store or Lopa targeting an Asiddha address ($\geq 0x50$) outside an Adhikāra block is rejected at compile time with `ParibhashaError("P4", "Asiddha-outside-scope")`, regardless of the ring level inferred. This closes a gap where a compiler without the Ring₀ auto-promotion bug might silently accept an out-of-scope Asiddha write at Ring₂.

A fifth bug class is prevented:

- **BUG-5 (P2):** Key zeroization is structural, not advisory. The compiler enforces that every `lopah` on an Asiddha address was preceded by a Store or Write on that same address. Erasure is not a separate advisory step—it is the proof of the prior write. A programmer cannot omit it without triggering a P2 rejection.

The pipeline produces 10 ABI words (40 bytes). Part A (14 compiler-level ABI structural checks) verifies: correct bookend structure, balanced Adhikāra scope, Ring₀ on all Asiddha ops (both Store and Lopa), `COMP = 1` on Anuvṛtti words, `FLAGS = 0xE` on the Sandhi-first word, and absence of Lopa outside scope. Part B (12 UART checks) confirms firmware execution: `ADHIKARA-OPEN`, `SANDHI-FIRST`, `SANDHI-COMMIT`, `ZEROIZED`, `ADHIKARA-CLOSE`, and the `SPRINT13 RESULT: ALL CHECKS PASSED` sentinel. Two Paribhāṣā rejection checks confirm that illegal programs (Asiddha Store outside Adhikāra; Lopa on never-written target) are refused at compile time. All 28 checks pass.

Compiler changes are minimal. Two lines suffice: extending the Ring₀ auto-promotion predicate from a single opcode to a two-element set, and adding a six-line P4b guard in `validate_ir()`. No changes to the ABI format, the firmware runtime, or the Sañjñā resolver are required. This is consistent with the earlier observation that structural guarantees are domain-agnostic: the Sprint 13 scenario is entirely new, but the enforcement machinery already existed; only its coverage needed to be closed.

6 Structural vs. Warning-Based Enforcement

The central claim of this paper can be stated precisely: the programs rejected by P1–P4 and S1–S5 have *no representation* in the PSL ABI. This is stronger than a warning in three ways.

Suppression. A compiler warning can be disabled. A MISRA deviation can be documented and approved. A Frama-C annotation can be omitted. In PSL, the rejected program has no binary representation—there is nothing to suppress.

Completeness. Static analysis tools operate on programs that exist. PSL’s type system prevents the programs from existing. The distinction matters for certification: a tool that reports “no violations found” on a program that has already been filtered by the type system provides a stronger guarantee than a tool applied to arbitrary C.

Audit boundary. In C, a Ring₀ write may appear anywhere in the source—finding all such writes requires whole-program analysis. In PSL, every Ring₀ instruction is lexically contained within an `adhikarah` block. The audit boundary is structural and source-visible.

Table 3 summarises the C burden vs. PSL structural guarantee for each rule.

Table 3: C programmer burden vs. PSL structural guarantee per rule.

Rule	C: programmer must	PSL: structurally guaranteed
P1 Anuvṛtti-at-start	Track whether prior write exists (global state)	Compiler tracks <code>prev_target</code> ; Anuvṛtti-at-start is ill-formed
P2 Lopa-on-unwritten	Track written-set per address (runtime or convention)	Compiler tracks W ; Lopa on $a \notin W$ is rejected
P3 Vṛddhi-on-Siddha	Check address range before Ring_0 write	Siddha/Asiddha is a compile-time partition; Ring_0 on Siddha is a category error
P4 Ring₀-outside-scope	Ensure privilege mode; typically OS-enforced	Ring_0 requires lexical <code>adhikarah</code> block; absent otherwise
S1–S5 Sandhi	Assert atomicity by convention; memory barriers at runtime	Five coherence rules checked at compile time; <code>FLAGS = 0xE</code> is structural
Saṅjñā	<code>#define</code> or <code>const</code> ; correctness by convention	Two-pass resolution; binary identity verified per sprint

7 Limitations and Future Work

Scope of the type system. PSL prevents four structural error classes. It does not prevent all embedded bugs. Functional correctness (“does the valve open to the right pressure?”) is not addressed. Information-flow security beyond privilege-region separation is not addressed. PSL is a complement to, not a replacement for, full formal verification.

Address space. The current implementation uses 8-bit device addresses ($[0x00, 0xFF]$) with a single partition threshold. Real embedded systems have richer address maps. Extending PSL to multi-region, multi-threshold address spaces is straightforward in principle but requires extending the P3/P4 predicates and the Saṅjñā resolver.

Hardware target. All proofs use QEMU virt, which faithfully simulates RV32 MMIO and UART. The proofs have not been run on physical silicon. FPGA or microcontroller deployment is the natural next step; the deterministic binary produced by PSL makes such deployment straightforward.

Compiler implementation. The compiler is implemented in Python 3.10 (832 lines). Sprint 14 establishes a five-phase CompCert-style verified chain [8] with 47 machine-checked assertions; Sprint 15 provides a Lean 4 formal proof [3] of `compile_sound` by structural induction (zero `sorry`, one named axiom). The one remaining gap—the forward-simulation lemma for P2—is verified empirically but not yet mechanically closed. A certified compiler would close it with a full Lean 4 or Coq proof.

Future work. Immediate extensions include: (a) an FPGA deployment proof on physical hardware; (b) extension of the Sandhi mechanism to multi-instruction atomic sequences; (c) a web playground enabling interactive experimentation with PSL source and ABI output; (d) a full mechanical proof of the `p2_runtime_correctness` forward-simulation lemma in Lean 4, which would make the entire `compile_sound` proof axiom-free; (e) extension of the Lean 4 model to cover the Sandhi fusion rules S1–S5 and the Saṅjñā two-pass resolver.

8 Conclusion

We have presented PSL, a domain-specific language for embedded peripheral control whose type system is derived from Pānini’s Pāribhāṣā meta-rules. The central claim—that a class of em-

bedded programming errors is made structurally inexpressible by the type system, and that this transfers faithfully to real hardware execution—is supported by 206 independently reproducible checks across fifteen proof sprints: 153 freestanding RV32 firmware checks over thirteen hardware sprints, a 5 000-program differential test suite, three real-world domain demonstrations (secure MMIO boot sequencing, safety-critical valve interlock, and cryptographic key lifecycle management), a five-phase CompCert-style verified compilation chain [8], and a Lean 4 formal proof of `compile_sound_statement` by structural induction [3].

The key observation is that Pānini’s mechanism for making certain Sanskrit derivations impossible to construct—not ill-formed at parse time, but impossible to derive—maps cleanly to the problem of making certain embedded instruction sequences impossible to compile. The structural guarantee is not a new idea; it is 2 400 years old. It has simply not previously been applied to instruction-set type systems with hardware proofs.

A warning can be suppressed. A structural impossibility cannot.

Acknowledgements

The author thanks the QEMU and GNU/RISC-V toolchain maintainers whose open-source infrastructure made this work reproducible without institutional resources.

References

- [1] George Cardona. *Pānini: His Work and Its Traditions*, volume 1. Motilal Banarsidass, Delhi, 1988.
- [2] Pascal Cuoq, Florent Kirchner, Nikolai Kosmatov, Virgile Prevosto, Julien Signoles, and Boris Yakobowski. Frama-C: A software analysis perspective. In *Software Engineering and Formal Methods (SEFM)*, pages 233–247, 2012.
- [3] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Artificial Intelligence*, pages 625–635. Springer, 2021.
- [4] Peter Zilahy Ingerman. “pānini–Backus form” suggested. *Communications of the ACM*, 10(3):137, 1967.
- [5] John Kadvany. *Pānini’s Grammar and Its Computational Insights*. History and Philosophy of Logic, 2016. Vol. 37, No. 4, pp. 325–346.
- [6] Sumitra Mangesh Katre. *Ashtadhyayi of Panini*. Deccan College Post-Graduate and Research Institute, Pune, 1987. Transliteration and translation of all 3959 sutras.
- [7] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. seL4: Formal verification of an OS kernel. In *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles (SOSP)*, pages 207–220, 2009.
- [8] Xavier Leroy. Formal verification of a realistic compiler. In *Communications of the ACM*, volume 52, pages 107–115, 2009.
- [9] Nicholas D. Matsakis and Felix S. Klock. The Rust language. *ACM SIGAda Ada Letters*, 34(3):103–104, 2014.

- [10] John W. McCormick and Peter C. Chapin. *Building High Integrity Applications with SPARK*. Cambridge University Press, 2015.
- [11] Andrei Sabelfeld and Andrew C. Myers. Language-based information-flow security. *IEEE Journal on Selected Areas in Communications*, 21(1):5–19, 2003.